Concepts

1. Recursion Fundamentals ⭐

```
static void solve(int n) {
   if (n == 0) return;     // base case(Common mistake: Forgetting base case)

   solve(n - 1);           // recursive call
}
```

Why needed
All problems A–H use recursion directly or indirectly.
Merge Sort, Tree building, Fast Power → all recursive.

Key rule
Every recursive function MUST have:

- Base case
- Smaller subproblem
- Combination step

---

2. Divide and Conquer Structure ⭐

```
// Divide
int mid = (left + right) / 2;

// Conquer
solve(left, mid);
solve(mid + 1, right);

// Combine
merge(left, mid, right);
```

Why needed
Problem A & B → Merge Sort
Problem F → Balanced BST order
Problem G & H → Tree reconstruction

Core idea
Split the problem into halves repeatedly.

---

3. Merge Sort Implementation (From Scratch) ⭐⭐⭐

```
static void mergeSort(int[] arr, int left, int right) {
   if (left >= right) return;    // base case

   int mid = (left + right) / 2;

   mergeSort(arr, left, mid);
   mergeSort(arr, mid + 1, right);

   merge(arr, left, mid, right);
}
```

Why needed
Problem A → count inversions
Problem B → modified inversion count

Important concept
Count during merging, NOT after sorting.

---

4. Counting During Merge ⭐⭐⭐

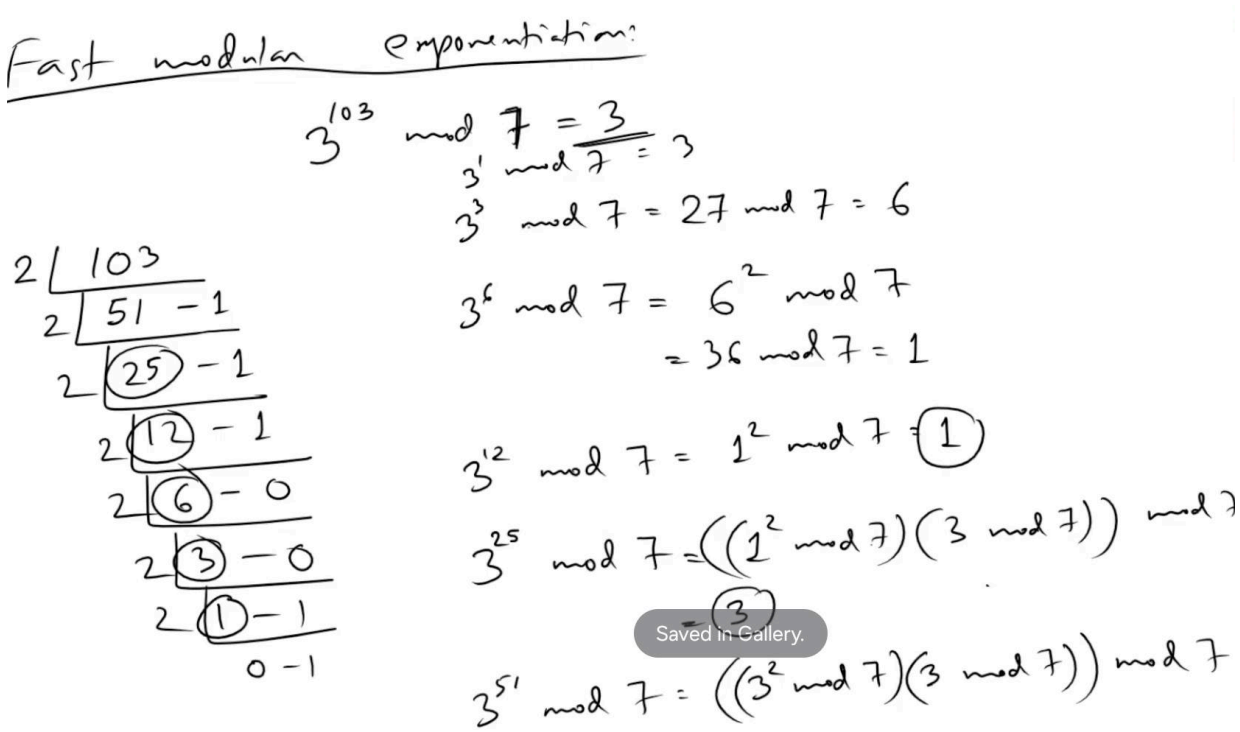
```
if (leftArr[i] <= rightArr[j]) {
   arr[k++] = leftArr[i++];
} else {
   arr[k++] = rightArr[j++];
   inversions += (mid - i + 1);   // all remaining left are greater
}
```

Why needed
This is the core trick of Problem A.

Big idea
We avoid O(n²) by counting blocks of elements.

---

5. Binary Exponentiation (Fast Power) ⭐⭐⭐



Saved in Gallery.

```
a = a % mod;    // reduce base first

        while (b > 0) {
            if (b % 2 == 1) {
                result = (result * a) % mod;
            }

            a = (a * a) % mod;    // square the base  _ _ _ _ _ _ _
            b = b / 2;            // divide exponent by 2(Koy bar korbo)
        }
```

```
103 = 1100111 (binary)

That means:

103= 64+32+4+2+1
So actually:
3^103= 3^64×3^32×3^4×3^2×3^1
```

Why needed
Problem C → a^b mod M
Problem D → Matrix power
Problem E → Geometric series

Core idea

We don't use `Math.pow()` because:

- It uses double
- It loses precision
- Cannot handle huge powers properly

Even/odd formulas exist because:

- Even exponent can be halved directly
- Odd exponent must remove one `a` first

`MOD` is:

- A large number (usually 1e9+7)
- Used to keep values small and prevent overflow

Reduce exponent by half each step → O(log b)

---

6. Modular Arithmetic Rules ⭐⭐⭐

```
(a + b) % M
(a * b) % M
```

Why needed
Problems C, D, E require mod 1e9+7

Important rules

```
Addition:

(a + b) % M = ((a % M) + (b % M)) % M

Multiplication:

(a * b) % M = ((a % M) * (b % M)) % M
```

Very common mistake
Forgetting modulo after multiplication → overflow.

---

7. StringBuilder Basics ⭐

```
StringBuilder sb = new StringBuilder();

sb.append("Hello").append(" ").append("World").append("\n");
System.out.println(sb.toString());
```

Why needed
Using `+` in loops creates many temporary String objects → slow and memory-heavy.
StringBuilder allows **efficient string concatenation** by modifying the same object.

Key rules

- Create `StringBuilder` once before the loop.
- Use `append()` for all additions instead of `+`.
- Convert to String with `sb.toString()` **once at the end** for output.
- Ideal for problems with **lots of output** or **large strings**, e.g., printing matrices in loops, building answers for contests.

---

8. Matrix Exponentiation Pattern ⭐⭐⭐

```
static long[][] matrixPower(long[][] base, long exp) {
   long[][] result = identityMatrix();

   while (exp > 0) {
       if (exp % 2 == 1)
           result = multiply(result, base);

       base = multiply(base, base);
       exp /= 2;
   }

   return result;
}
```

Why needed
Problem D
Also future DP optimizations (Fibonacci etc.)

---

9. Geometric Series Using Divide & Conquer ⭐⭐⭐

We compute:

$S(n) = a^1 + a^2 + ... + a^n$

Key identity:

If n even:

$S(n) = S(n/2) * (1 + a^{(n/2)})$

Why needed
Problem E

Big idea
Split series in half instead of looping n times.

---

10. Tree Traversal Logic ⭐⭐⭐

Preorder: Root Left Right

Inorder: Left Root Right

Postorder: Left Right Root

---

11. Reconstructing Tree Using Index Ranges ⭐⭐⭐

Instead of creating actual TreeNode objects,
we pass index ranges:

```
build(inStart, inEnd, preStart, preEnd);
```

Why needed
Problem G & H

Important concept
Use indices instead of slicing arrays (faster).

---

12. Balanced BST Construction From Sorted Array ⭐⭐

Pick middle first:

```
int mid = (left + right) / 2;
```

```
print(arr[mid]);

solve(left, mid - 1);
solve(mid + 1, right);
```

Why needed
Problem F

Idea
This generates a preorder of minimum-height BST.

---

13. Bitwise Right Shift (b >>= 1) ⭐

```
b >>= 1;    // equivalent to b = b / 2
b%1         //equivalent to b%2==1
```

Why needed

- Used in **fast exponentiation** to divide the exponent by 2 each step
- Makes exponentiation O(log n) instead of O(n).

More topics needed (Covered in Lab 1 & 2):

- Imports / Packages
- Input / Output (BufferedReader, PrintWriter)
- String → Number Parsing (int, long, double)
- StringTokenizer Usage
- String Splitting (.split)
- Collections.sort()
- Comparator Concept (sorting logic)
- Character Utility Methods
- String Helper Methods
- ArrayList Basics & Operations
- Storing Pairs (int[])

- Loops (standard & enhanced for-loop)
- Math Helper Methods
- Sliding Window Pattern
- Subarray Concept
- Lower Bound / Upper Bound Functions
- HashMap Basics
- Binary Search Template
- Two pointer

Tab 2

A. Count the Inversion

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: sort, open, file, creat(, fstream, thread, process, system(, exec(

Here is a Pseudocode of the Merge Sort Algorithm.

Now, you are given an array ***A*** of size ***N*** of ***N*** distinct integers. It is guaranteed that the array A contains a permutation of integers from 1 to N (i.e., every integer from 1 to N appears exactly once).

1. Count the number of inversions in the given array.
2. Sort the array in non-decreasing order.

An inversion is a pair (i, j) where $i < j$ and $A[i] > A[j]$.

Examples

input Copy

```
5
1 2 5 4 3
```

output Copy

```
3
1 2 3 4 5
```

```
import java.util.*;
import java.io.*;

public class A{
    static long inversion=0;

    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        int n= Integer.parseInt(br.readLine());
        String s= br.readLine();
        StringTokenizer st=new StringTokenizer(s, " ");
        int[] arr=new int[n];

        //filling up the array
        for(int i=0; i<n; i++){
            arr[i]= Integer.parseInt(st.nextToken());
        }

        mergeSort(arr, 0, n-1);

        out.println(inversion);
        for(int i=0; i<n; i++){
            out.print(arr[i] + " ");
```

```
    }
    out.println();
    out.flush();
}


public static void mergeSort(int[] arr, int l, int r){
    if(l<r){
        int mid=(l+r)/2;

        mergeSort(arr, l, mid);
        mergeSort(arr, mid+1, r);

        merge(arr, l, mid, r);
    }
}



public static void merge(int[] arr, int l, int mid, int r){
    int n1=mid-l+ 1;
    int n2= r-mid;

    int[] arr1= new int[n1];
    int[] arr2= new int[n2];

    for(int i=0; i<n1; i++) arr1[i]= arr[l+ i ];   //* */
    for(int i=0; i<n2; i++) arr2[i]= arr[mid+1 + i];

    int i=0, j=0, k=l;

    while(i<n1 && j<n2){
        if(arr1[i]< arr2[j]) arr[k++] = arr1[i++];

        else if(arr2[j]< arr1[i]){
            arr[k++] = arr2[j++];
            inversion+=(n1-i);        //Inversion r jonno merge sort bhalo
        }
    }

    while(i<n1) arr[k++]=arr1[i++];
    while(j<n2) arr[k++]=arr2[j++];
}
```

}

//Inversion hocche koyta value of L, oi R r value r age boshar kotha chilo

---

B. Count the Inversion Revisited

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given an array A of N integers. Find the number of pairs of indexes (i, j) such that $i < j$ and $A[i] > A[j]^2$.

Input

The first line contains an integer N $(1 \leq N \leq 10^5)$ — denoting the length of the list.

In the next line, there will be N integers $a_1, a_2, a_3 \ldots a_n$ $(-10^6 \leq a_i \leq 10^6)$ separated by spaces.

Output

Output a single integer - number of such pairs.

Examples

input Copy

```
6
10 2 5 1 -2 25
```

output Copy

```
6
```

input Copy

```
5
5 4 3 -2 -1
```

output Copy

```
4
```

```
import java.util.*;
import java.io.*;

public class B{

   public static void main(String[] args) throws IOException{
      BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
      PrintWriter out= new PrintWriter(System.out);

      int n= Integer.parseInt(br.readLine());
      String s= br.readLine();
      StringTokenizer st=new StringTokenizer(s, " ");
      long[] arr=new long[n];
```

```
    //filling up the array
    for(int i=0; i<n; i++){
        arr[i]= Long.parseLong(st.nextToken());
    }

    long result= mergeSort(arr, 0, n-1);

    out.println(result);
    out.flush();
}


public static long mergeSort(long[] arr, int l, int r){
    if(l>=r) return 0;

    else{
        int mid=(l+r)/2;
        int count=0;

        count+= mergeSort(arr, l, mid);
        count+= mergeSort(arr, mid+1, r);

        count+= merge(arr, l, mid, r);

        return count;
    }
}



public static long merge(long[] arr, int l, int mid, int r){
    if(l>=r) return 0;

    int n1=mid-l+ 1;
    int n2= r-mid;
    int count=0;

    long[] arr1= new long[n1];
    long[] arr2= new long[n2];

    for(int i=0; i<n1; i++) arr1[i]= arr[l+ i];
    for(int i=0; i<n2; i++) arr2[i]= arr[mid+1 + i];
```

```
    //count part (Binary search)
    for(int j = 0; j < n2; j++){
    long target = (long)arr2[j] * arr2[j];
    int low = 0, high = n1;
    while(low < high){
        int mid_1 = (low + high) / 2;
        if(arr1[mid_1] > target) high = mid_1;
        else low = mid_1 + 1;
    }
    count += n1 - low;  // all elements from low to n1-1 satisfy arr1[i] > arr2[j]^2
    }


    //sort part
    int i=0, j=0, k=l;
    while(i<n1 && j<n2){
        if(arr1[i]<= arr2[j]) arr[k++] = arr1[i++];

        else{
            arr[k++] = arr2[j++];
        }
    }

    while(i<n1) arr[k++]=arr1[i++];
    while(j<n2) arr[k++]=arr2[j++];
    return count;
  }

}
```

C. Fast Power Drift

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: pow, open, file, creat(, fstream, thread, process, system(, exec(

You are given two integers ***a*** and ***b***. Calculate $a^b \ mod \ 107$.

Input

The input file contains two integers ***a*** $(1 \le a \le 10^4)$ and ***b*** $(1 \le b \le 10^{12})$.

Output

Print one integer — the result of $a^b \ mod \ 107$.

Examples

input Copy

```
100 3
```

output Copy

```
85
```

- a == 3^0, 3^1, 3^2, 3^3, 3^4, 3^5, 3^6……(_____ egular power)
- b == Determine the number of times the iteration will work + see if odd or even
- result == multiplied value

```
import java.util.*;
import java.io.*;

public class C{

    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        String s= br.readLine();
        StringTokenizer st=new StringTokenizer(s, " ");
        long a=Long.parseLong(st.nextToken());
        long b=Long.parseLong(st.nextToken());

        out.println(modularPower(a,b, 107));
        out.flush();
    }


    static long modularPower(Long a, Long b, int mod){
        long result=1;
        a=a%mod;

        while(b>0){
```

```
        if(b%2==1){
            result = (result * a)% mod;
        }
        a=(a*a)%mod;
        b=b/2;
    }

    return result;
  }
}
```

D. Fast Matrix Drift

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given a 2×2 integer matrix A and an integer exponent X $(1 \le X \le 10^9)$.

Your task is to compute the matrix A^X, where all intermediate operations (additions and multiplications during matrix multiplication) are performed modulo $10^9 + 7$.

Formally, for two matrices:

$$B = \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}, \quad C = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}.$$

Their product $D = B \times C$ is defined as: $D_{ij} = (b_{i1} \cdot c_{1j} + b_{i2} \cdot c_{2j}) \bmod (10^9 + 7)$.

Input

The first line contains a single integer T $(1 \le T \le 10^5)$ — the number of test cases.

The first line of each test case contains four integers a_{11}, a_{12}, a_{21}, and a_{22} where $0 \le a_{ij} \le 10^9$ — the elements of the matrix A. The second line contains an integer X where $1 \le X \le 10^9$.

Output

For each test case, print two lines, each containing two integers: the resulting 2×2 matrix A^X. Each element should be printed modulo $10^9 + 7$.

Examples

input Copy

```
2
1 1 1 0
5
2 0 0 2
3
```

output Copy

```
8 5
5 3
8 0
0 8
```

```
import java.util.*;
import java.io.*;
```

```
public class D{

    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);
       StringBuilder sb=new StringBuilder();

        int n=Integer.parseInt(br.readLine());
        for(int i=0; i<n; i++){
            //Fill the array
            String s= br.readLine();
            StringTokenizer st=new StringTokenizer(s, " ");
            long[][] arr=new long[2][2];

            arr[0][0]=Long.parseLong(st.nextToken());
            arr[0][1]=Long.parseLong(st.nextToken());
            arr[1][0]=Long.parseLong(st.nextToken());
            arr[1][1]=Long.parseLong(st.nextToken());


            //take the power
            long power=Long.parseLong(br.readLine());
            long[][] result= modularPower(arr, power);


            //print
           sb.append(result[0][0]).append(" ").append(result[0][1]).append("\n");
            sb.append(result[1][0]).append(" ").append(result[1][1]).append("\n");
        }
        out.print(sb.toString());
        out.flush();




    static long[][] modularPower(long[][] arr, long power){
        long[][] result={{1,0}, {0,1}};                    //identity matrix
        long[][] a=arr;

        while(power>0){
            if(power%2==1){
                result = multiply(result, a);
            }
```

```
        a=multiply(a,a);
        power=power/2;
    }

    return result;
  }



  static long MOD= 1000000007;
  static long[][] multiply(long[][] A, long[][] B){

    long a00= (A[0][0] * B[0][0] % MOD + A[0][1] * B[1][0] % MOD) % MOD;
    long a01 = (A[0][0] * B[0][1] % MOD + A[0][1] * B[1][1] % MOD) % MOD;
    long a10 = (A[1][0] * B[0][0] % MOD + A[1][1] * B[1][0] % MOD) % MOD;
    long a11 = (A[1][0] * B[0][1] % MOD + A[1][1] * B[1][1] % MOD) % MOD;

    A[0][0]= a00;
    A[0][1]= a01;
    A[1][0]= a10;
    A[1][1]= a11;

    return A;
  }
}
```

E. Fast Series Drift

time limit per test: 2.5 seconds
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

You are given three integers a, n and m. Calculate $(a^1 + a^2 + \ldots + a^n) \% m$.

Input

The first line contains an integer T $(1 \le T \le 10^5)$ — total numbers of test cases.

In each of the next T test cases, there are three integers a $(1 \le a \le 10^6)$, n $(1 \le n \le 10^{12})$ and $(1 \le m \le 10^9)$

Output

Print one integer — the result of $(a^1 + a^2 + \ldots + a^n) \% m$.

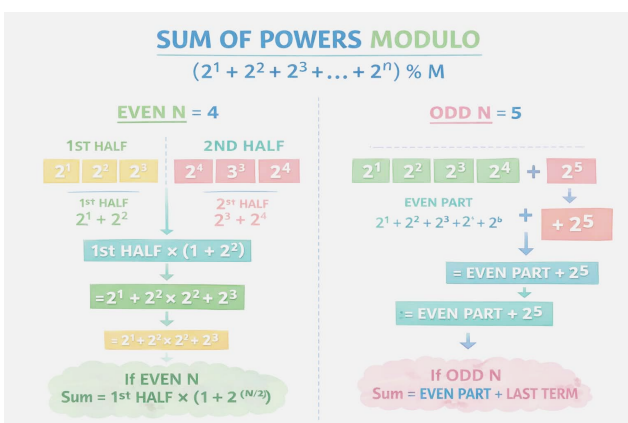
Example

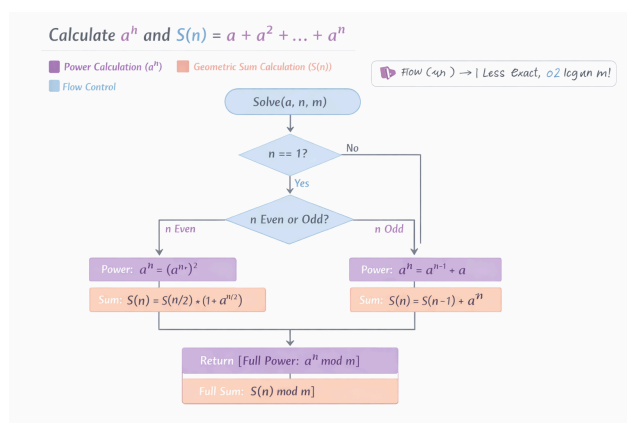
input Copy

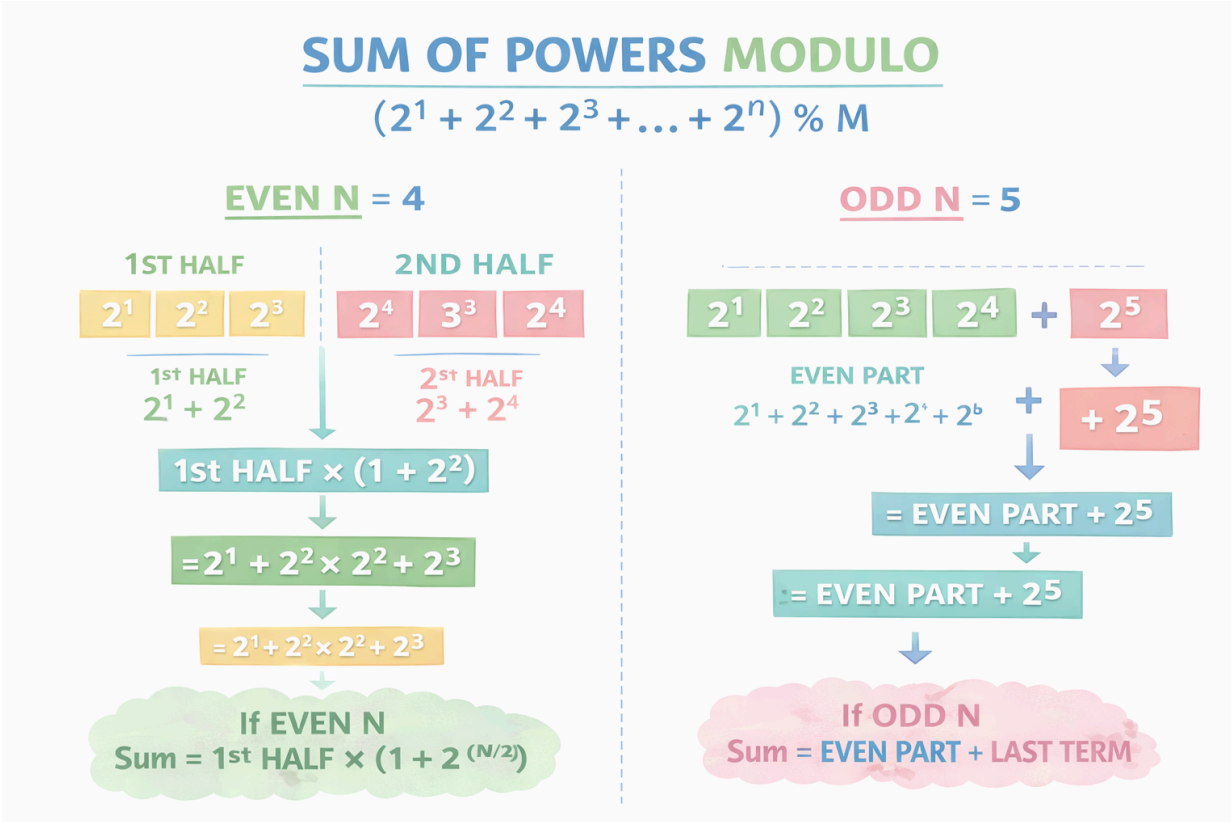
```
3
2 5 1000
2 9 1000
1 100 30
```

output Copy

```
62
22
10
```








```
import java.util.*;
import java.io.*;

public class E{

    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        int t=Integer.parseInt(br.readLine());
        for(int i=0; i<t; i++){
            String s= br.readLine();
            StringTokenizer st=new StringTokenizer(s, " ");
            long a=Long.parseLong(st.nextToken());
            long n=Long.parseLong(st.nextToken());
            long m=Long.parseLong(st.nextToken());


            //Series sum
            if(a==1) out.println(n%m);
            else{
```

```
            long[] ans=geometricSum(a,n,m);
            out.println(ans[1]);
        }
    }
    out.flush();
}


static long[] geometricSum(long a, long n, long m) {
    if (n == 1) return new long[] {a % m, a%m};

    if (n % 2 == 0) {
        long[] half = geometricSum(a, n/2, m);

        long pow_half = half[0];     //a^n % m
        long sum_half = half[1];     //S(n) % m    where S(n) = a + a² + ... + aⁿ

        long fullPow = (pow_half * pow_half) % m;   //half power multiply korle full power ashe
        long fullSum = (sum_half + (pow_half * sum_half) % m) % m;

        return new long[]{fullPow, fullSum};

    } else {
        long[] prev = geometricSum(a, n - 1, m);

        long pow = (prev[0] * a) % m;
        long sum = (prev[1] + pow) % m;

        return new long[]{pow, sum};
    }
}



static long power(long a, long b, long mod){
    long result=1;
    a=a%mod;

    while(b>0){
        if((b&1)==1){
            result = (result * a)% mod;
        }
        a=(a*a)%mod;
        b>>=1;
```

```
        }

        return result;
    }
}
```

F. Ordering Binary Tree

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

you are given an array ***A*** of size ***N*** in **increasing** order. Find an order of these N integers such that, if these integers are inserted into a Binary Search Tree (BST) one by one, the height of the resulting BST is minimized.

A Binary Search Tree is a binary tree in which each node has at most two children, referred to as the left and right child. For any node, all elements in the left subtree are smaller than the node's value, and all elements in the right subtree are greater than the node's value.

The height of a Binary Search Tree is defined as the maximum depth among all the nodes in the tree.

Note: All the elements in the array A are guaranteed to be unique. In other words, $A_i \neq A_j$ if $i \neq j$.

Input

The first line contains an integer N $(1 \leq N \leq 10^5)$ — denoting the length of the list.

In the next line, there will be N integers $a_1, a_2, a_3 \ldots a_n$ $(1 \leq a_i \leq 10^9)$ in non-descending order separated by spaces.

Output

Output the order of the elements such that when inserted into a Binary Search Tree, the height of the tree is minimized. If there are multiple such orders then find any of them.

Example

input Copy

```
5
1 2 3 4 5
```

output Copy

```
3 1 2 4 5
```

```
import java.util.*;
import java.io.*;

public class F{
    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);

        int n=Integer.parseInt(br.readLine());
        String s= br.readLine();
```

```
        StringTokenizer st=new StringTokenizer(s, " ");
        int[] arr=new int[n];
        for(int i=0; i<n; i++){
            arr[i]= Integer.parseInt(st.nextToken());
        }

        int[] result=new int[n];
        BST_inorder(arr, 0, n-1, result);
        for(int i=0; i<n; i++) out.print(result[i]+ " ");

        out.flush();
    }




    static int indx=0;
    static void BST_inorder(int[] arr, int l, int r, int[] result){
        if(l>r) return;     //Base case

        int mid=(l+r)/2;
        result[indx++]=arr[mid];
        BST_inorder(arr, l, mid-1, result);
        BST_inorder(arr, mid+1, r, result);
    }
}
```

G. 220 Trees

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

There is a Binary Tree with ***N*** nodes. You are given the in–order and pre-order traversals of the tree. Your task is to determine the post-order traversal of the tree.

Input

The first line contains an integer N $(1 \leq N \leq 1000)$ — the number of nodes in the binary tree.

In the next line, there will be N integers $a_1, a_2, a_3 \ldots a_n$ $(1 \leq a_i \leq N)$ separated by spaces – representing the in-order traversal of the tree.

The following line, there will be N integers $b_1, b_2, b_3 \ldots b_n$ $(1 \leq b_i \leq N)$ separated by spaces – representing the pre-order traversal of the tree.

Output

Print N space-separated integers representing the post-order traversal of the binary tree.

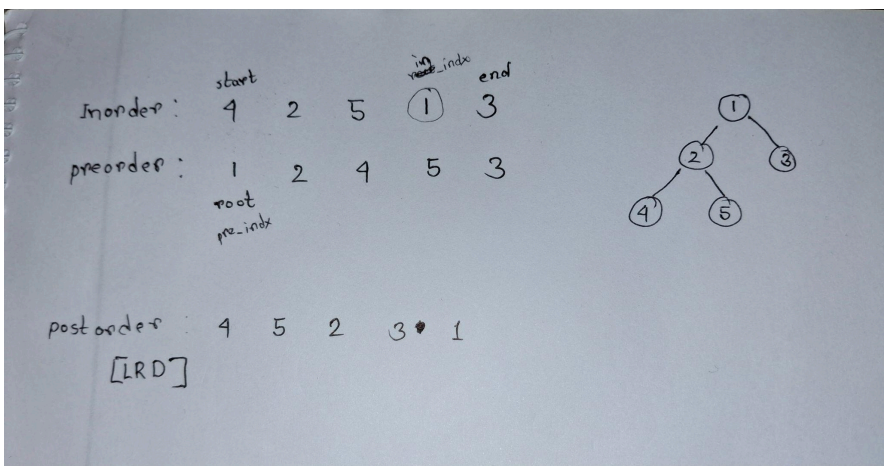
Example

input Copy

```
5
4 2 5 1 3
1 2 4 5 3
```

output Copy

```
4 5 2 3 1
```



//Inorder r kaj only bhag kora

```
import java.util.*;
import java.io.*;

public class G{

    static int[] inorder;
    static int[] preorder;
    static int pre_indx=0;
```

```
    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);


        //Filling up the array
        int n=Integer.parseInt(br.readLine());
        String s_in= br.readLine();
        StringTokenizer st_in=new StringTokenizer(s_in, " ");
        String s_pre= br.readLine();
        StringTokenizer st_pre=new StringTokenizer(s_pre, " ");
        inorder=new int[n];
        preorder= new int[n];
        for(int i=0; i<n; i++){
            inorder[i]= Integer.parseInt(st_in.nextToken());
            preorder[i]=Integer.parseInt(st_pre.nextToken());
        }


        postorder(0, n-1, out);
        out.flush();
    }

    static void postorder(int start, int end, PrintWriter out){
        if(start>end) return;   //base case

        int root= preorder[pre_indx++];
        int in_indx= -1;

        for(int i=start; i<=    end; i++){
            if(inorder[i]==root){
                in_indx=i;
                break;
            }
        }

        //Left-Right-Data
        postorder(start, in_indx-1, out);
        postorder(in_indx+1, end, out);
        out.print(root+ " ");
    }
}
```

H. 220 Trees Reassessed

time limit per test: 1 second
memory limit per test: 256 megabytes

Forbidden Keywords for the Quiz: open, file, creat(, fstream, thread, process, system(, exec(

There is a Binary Tree with ***N*** nodes. You are given the in-order and post-order traversals of the tree. Your task is to determine the pre-order traversal of the tree.

Input

The first line contains an integer N $(1 \le N \le 1000)$ — the number of nodes in the binary tree.

In the next line, there will be N distinct integers $a_1, a_2, a_3 \ldots a_n$ $(1 \le a_i \le N)$ separated by spaces – representing the in-order traversal of the tree.

The following line, there will be N distinct integers $b_1, b_2, b_3 \ldots b_n$ $(1 \le b_i \le N)$ separated by spaces – representing the post-order traversal of the tree.

Output

Print N space-separated integers representing the pre-order traversal of the binary tree.

Examples

input Copy

```
5
1 2 3 4 5
1 4 5 3 2
```

output Copy

```
2 1 3 5 4
```

```
import java.util.*;
import java.io.*;

public class H{

    static int[] inorder;
    static int[] postorder;
    static int post_indx=0;


    public static void main(String[] args) throws IOException{
        BufferedReader br= new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out= new PrintWriter(System.out);


        //Filling up the array
        int n=Integer.parseInt(br.readLine());
```

```
        String s_in= br.readLine();
        StringTokenizer st_in=new StringTokenizer(s_in, " ");
        String s_post= br.readLine();
        StringTokenizer st_post=new StringTokenizer(s_post, " ");
        inorder=new int[n];
        postorder= new int[n];
        for(int i=0; i<n; i++){
            inorder[i]= Integer.parseInt(st_in.nextToken());
            postorder[i]=Integer.parseInt(st_post.nextToken());
        }


        preorder(0, n-1, 0, n-1, out);
        out.flush();
    }

    static void preorder(int in_start, int in_end, int post_start, int post_end, PrintWriter out){
        if(in_start>in_end || post_start>post_end) return;   //base case

        int root= postorder[post_end];   //* */

        //Data-left-right
        out.print(root+ " ");
        int in_indx= -1;
        for(int i=in_start; i<=  in_end; i++){
            if(inorder[i]==root){
                in_indx=i;
                break;
            }
        }

        int leftSize = in_indx - in_start;

        preorder(in_start, in_indx-1, post_start, post_start+leftSize-1, out);
        preorder(in_indx+1, in_end, post_start+leftSize, post_end-1   , out);

    }
}
```